

Data Warehouse Optimization – report

1. Aim of the laboratory

The aim of the task is to show issues concerning various physical cube models and aggregation design.

2. Preliminary assumptions

Size of the database (data warehouse): 144.00MB

Number of rows in monitoring fact table (our main): 3179

Testing environment:

Measurements were taken on HP Pavilion Laptop 15-eh3xxx equipped with an AMD Ryzen 5 7530U with Radeon Graphics processor, 16GB of RAM and 512GB of Internal Storage. We used Windows 11 via Boot Camp Assistant (the specifications remained the same). To evaluate the processing time of a cube, we used SQL Server Management Studio with extension SQL Server Profiler. During the measurements, the only active applications on the laptop were SSMS and a web browser with opened instructions and example report, and Visual Studio 2022.

3. Theoretical Assumptions

	MOLAP	HOLAP	ROLAP
Querying time	Short	Moderate (short with well designed aggregations)	Long
Processing time	Long	Moderate (if no aggregations are designed, it will be short)	Short
Total size	Big (size of the measure group is much smaller if no aggregations are designed for them)	Moderate	Small

4. Testing

Testing query execution times for different models, with and without defined aggregations. Testing cube processing times in the same testing settings.

Brief description of the queries:

1. (one with aggregations on dates)

Query that calculates cost of meal wastes due to leftovers for past 3 months

```
WITH
  MEMBER [Measures].[MealLossChange] AS
    ([Date].[Month].[May], [Measures].[Meal loss]) -
    ([Date].[Month].[March], [Measures].[Meal loss]),
  FORMAT_STRING = "Currency"
SELECT
  {
    [Measures].[Meal loss],
    [Measures].[MealLossChange]
  } ON COLUMNS,
  NON EMPTY [Meal Type].[Meal Type].Members ON ROWS
FROM [KinderGardenDW]
WHERE [Date].[Year].[2025]
```

2. (one for a particular dimension attribute)

Query that finds the most common ingredient that can be found in a meal with the highest waste level

```
WITH
  MEMBER [Measures].[Waste %] AS
    ROUND([Measures].[Waste Percentage], 2),
  FORMAT_STRING = "0.00%"
SELECT
  [Measures].[Waste %] ON COLUMNS,
  TopCount(
    CrossJoin(
      [Ingredient].[Name].[Name].Members,
      [Ingredient].[Type].[Type].Members,
      [Meal Recipe].[Name].[Name].Members
    ),
    1,
    [Measures].[Waste %]
  ) ON ROWS
FROM [KinderGardenDW]
```

3. (general one)

Query that compares total meals prepared vs. meals given vs. leftovers for each meal type in the past three months.

```

SELECT
    {[Measures].[Meal Prepared], [Measures].[Meal Given], [Measures].[Leftovers]} ON COLUMNS,
    [Meal Type].[Meal Type].MEMBERS ON ROWS
FROM [KinderGardenDW]
WHERE
{
    ([Date].[Year].[2025], [Date].[Month].[March]),
    ([Date].[Year].[2025], [Date].[Month].[April]),
    ([Date].[Year].[2025], [Date].[Month].[May])
}

```

To achieve optimal results of the processing time of a cube, we decided to take approximately 10 samples for each modification. The obtained results are presented in the following table.

Table 4.1. Processing time of cube and queries for MOLAP, HOLAP and ROLAP with and without aggregations

		Cube processing			Querie 1			Querie 2			Querie 3		
		Molap	Holap	Rolap	Molap	Holap	Rolap	Molap	Holap	Rolap	Molap	Holap	Rolap
Without Aggr		1595	2547	2663	42	229	261	34	186	201	13	143	99
		1512	1835	2478	43	211	206	43	203	203	9	130	105
		1308	2467	2756	49	322	324	25	205	206	10	126	139
		1658	2487	2714	37	246	304	21	199	200	14	128	142
		1489	2603	2495	45	249	226	25	168	184	13	122	92
		1376	2809	2494	43	250	251	20	199	185	9	134	128
		1401	2476	2511	43	228	203	17	185	191	16	111	138
		12527	2464	2364	46	233	235	26	200	179	13	128	101
		1453	2600	2550	50	298	219	27	164	214	12	112	127
		1477	2772	2488	44	325	270	27	200	209	10	127	90
With aggr													
		1553	1911	2513	43	217	249	26	197	207	13	139	129
		7578	2585	1829	45	229	246	26	187	202	15	106	132
		1387	2568	2635	49	234	238	32	202	193	10	136	140
		1469	2420	2681	45	241	314	26	205	203	14	135	129
		1570	2543	1960	43	301	239	25	159	176	13	133	111

1501	2566	2500	43	259	254	28	184	204	10	138	62
1443	2448	2606	37	315	264	39	190	202	11	108	137
1526	2602	2651	50	318	303	17	204	166	14	116	103
1429	2747	2870	45	257	252	23	208	180	13	139	138
1534	2507	2671	49	247	181	20	175	168	14	139	119

Afterwards, we decided to exclude outliers and calculate the mean and standard deviation for each column. The results are summarized in the following tables:

Tables 4.2. and 4.3. Mean and standard deviation of processing time of cube and queries for MOLAP, HOLAP and ROLAP with and without aggregations

	Cube processing			Querie 1			Querie 2			Querie 3		
Without Aggr	Molap	Holap	Rolap	Molap	Holap	Rolap	Molap	Holap	Rolap	Molap	Holap	Rolap
Mean	1474,33	2506,00	2551,30	44,20	259,10	249,90	26,50	190,90	197,20	11,90	126,10	116,10
SD	101,71	252,44	115,49	3,49	38,78	38,50	7,05	13,96	11,17	2,21	9,03	19,60

	Cube processing			Querie 1			Querie 2			Querie 3		
With Aggr	Molap	Holap	Rolap	Molap	Holap	Rolap	Molap	Holap	Rolap	Molap	Holap	Rolap
Mean	1490,22	2489,70	2491,60	44,90	261,80	254,00	26,20	191,10	190,10	12,70	128,90	120,00
SD	58,41	210,83	315,03	3,65	34,73	34,65	5,79	14,69	15,19	1,68	12,73	22,48

Table 4.4. Average time of processing cube and queries using MOLAP and ROLAP with and without aggregations

	MOLAP		HOLAP		ROLAP	
	Aggr.	No aggr.	Aggr.	No aggr.	Aggr.	No aggr.
Querying speed (for 3 different queries)	44,90	44,20	261,80	259,10	254,00	249,90
	26,20	26,50	191,10	190,90	190,10	197,2

	12,70	11,90	128,90	126,10	120,00	116,10
Processing time	1490,22	1474,33	2489,70	2506,00	2491,60	2551,30

5. Cache and aggregations settings

Aggregations:

Cube Objects	Default	Full	None	Unrestricted
Eaten Meal	8	1	1	0
Eaten Meal No	☐	☐	☐	☐
ID Date	☐	☐	☐	☐
ID Meal Type	☐	☐	☐	☐
ID Meal Recipe	☐	☐	☐	☐
ID Group Name	☐	☐	☐	☐
Meal Prepared	☐	☐	☐	☐
Meal Given	☐	☐	☐	☐
Leftovers	☐	☐	☐	☐
Meal Loss	☐	☐	☐	☐
Group Meal Price	☐	☐	☐	☐

Cache:

```
<ClearCache xmlns='http://schemas.microsoft.com/analysiservices/2003/engine'>
  <Object>
    <DatabaseID>KinderGarden</DatabaseID>
  </Object>
</ClearCache>
```

6. Discussion (comparison of the theory with the obtained results)

Processing the cube

	MOLAP	HOLAP	ROLAP
--	--------------	--------------	--------------

	Aggr.	No aggr.	Aggr.	No aggr.	Aggr.	No aggr.
Processing time	1490,22	1474,33	2489,70	2506,00	2491,60	2551,30

Based on the measured results, it can be observed that the **ROLAP model** consistently showed the highest average processing time both **with (2491,60 ms)** and **without aggregations (2551,30 ms)**. The **MOLAP model**, on the other hand, demonstrated the **shortest processing times**, averaging **1474,33 ms without aggregations** and **1490,22 ms with aggregations**. Interestingly, unlike theoretical assumptions which suggest that MOLAP should incur a higher cost due to data pre-loading, our observed data reveals that **MOLAP was the most efficient in cube processing time** in our case.

This discrepancy might be attributed to the relatively small size of our dataset (144 MB), where the benefits of data pre-loading in MOLAP outweigh the processing overhead. HOLAP showed intermediate performance, as expected, balancing between pre-aggregation and on-demand data retrieval.

Query execution

	MOLAP		HOLAP		ROLAP	
	Aggr	No aggr	Aggr	No aggr	Aggr	No aggr
Querying speed (for 3 different queries)	44,90	44,20	261,80	259,10	254,00	249,90
	26,20	26,50	191,10	190,90	190,10	197,2
	12,70	11,90	128,90	126,10	120,00	116,10

When it comes to query execution, our results align more closely with theory:

- **MOLAP delivered the fastest execution times** across all three queries, regardless of aggregation status. This is in line with its in-memory structure, allowing quick access to pre-processed data.
- **HOLAP and ROLAP**, both reliant on querying the relational database during execution, showed **significantly higher execution times**, particularly in **Query 1 and Query 2**, which involved more complex aggregation logic.

- For example, in Query 1 (related to meal waste over time), MOLAP averaged **44,20 ms** without aggregations, compared to HOLAP's **259,10 ms** and ROLAP's **249,90 ms**.

This pattern held across the other two queries, indicating that **pre-loading and storage strategy significantly impact runtime**, especially for analytic queries involving large result sets or aggregations.

Aggregations

	MOLAP		HOLAP		ROLAP	
	Aggr.	No aggr.	Aggr.	No aggr.	Aggr.	No aggr.
Processing time	1490,22	1474,33	2489,70	2506,00	2491,60	2551,30
Querying speed (for 3 different queries)	44,90	44,20	261,80	259,10	254,00	249,90
	26,20	26,50	191,10	190,90	190,10	197,2
	12,70	11,90	128,90	126,10	120,00	116,10

Defining aggregations influenced results less dramatically than expected. While **query execution times slightly decreased** with aggregations across all models (e.g., MOLAP Query 3: from 11,90 ms to 12,70 ms), the difference was not substantial.

However, a counterintuitive trend was observed in the **processing time**: in contrast to theoretical expectations that aggregations increase cube processing time, **MOLAP's processing time increased only slightly (from 1474,33 ms to 1490,22 ms)**. **ROLAP and HOLAP**, however, showed **reduced processing times** when aggregations were enabled, particularly notable in **ROLAP**, where the time dropped from **2551,30 ms to 2491,60 ms**.

This may suggest that the structure of the aggregations - possibly more aligned with the actual query dimensions - helped the engine streamline cube processing, especially in models not fully pre-loaded like ROLAP.

Conclusion

Our tests confirmed some theoretical assumptions while revealing dataset-specific anomalies:

- **MOLAP consistently outperformed HOLAP and ROLAP in both cube processing and query execution** in our testing environment.
- **Aggregations provided moderate benefits**, more evident in **query execution** than cube processing.
- **Storage model selection significantly impacts performance**, and should be tailored to the size of the dataset and the complexity of typical queries.

These results demonstrate the practical trade-offs in physical model selection and highlight the value of customized optimization for each data warehouse scenario.